

HTML5 — Formulaires & accessibilité

Développement web — Classe passerelle

Semaine 3 | **Module B**

Anne Terrier | cfpt·i — École d'informatique

Au programme aujourd'hui

1. À quoi sert un formulaire
2. Les champs de saisie
3. Les attributs qui encadrent la saisie
4. La validation native et ses limites
5. Structurer un formulaire
6. Accessibilité

Objectif de la semaine

Écrire des formulaires complets, corrects et utilisables par tous — y compris par une personne qui n'utilise ni souris ni écran.

Où en sommes-nous ?

Semaine 2 — afficher

Le MiniForum **présente** du contenu : titres, listes de sujets, messages. Tout est écrit en dur dans le HTML.

Semaine 3 — saisir

La page s'ouvre dans l'autre sens : le visiteur va pouvoir **entrer** quelque chose.

Dernière séance de HTML

Les formulaires écrits aujourd'hui sont ceux du MiniForum **final** : leur structure ne changera plus jusqu'à la soutenance. Ils resteront inertes quelques semaines — JavaScript les branchera en semaine 7, le serveur en semaine 15.

Le CSS commence la semaine prochaine.

1. À quoi sert un formulaire

Définition — Formulaire

Un **formulaire** est un ensemble de champs de saisie, regroupés dans une balise `<form>`, permettant à un visiteur de transmettre des données à l'application.

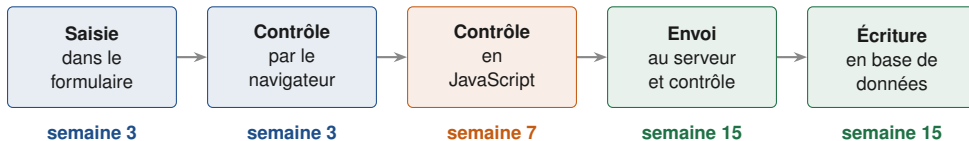
Sans formulaire

Le site **diffuse**. Le contenu vient du développeur, et de lui seul.

Avec formulaire

Le site **reçoit**. Un forum, une boutique, un réseau social n'existent que par ce que les visiteurs y déposent.

La vie d'une donnée saisie



Aujourd'hui : les **deux premières cases**. Les trois autres expliquent pourquoi certains choix faits ce matin paraîtront inutiles jusqu'en décembre.

La balise <form>

```
<form action="/topics" method="post">
  <!-- les champs de saisie viennent ici -->
  <button type="submit">Publier</button>
</form>
```

Attribut	Rôle
action	L'adresse à laquelle les données sont envoyées
method	La façon de les envoyer : get (dans l'URL) ou post (dans le corps de la requête)

get **ou** post ?

Ces deux attributs ne serviront qu'en semaine 15

Tant qu'aucun serveur n'écoute à l'adresse indiquée, `action` et `method` n'ont aucun effet visible : cliquer sur « Publier » recharge la page, et c'est tout.

get

Les données passent **dans l'URL**.

Pratique pour une recherche que l'on veut mettre en favori.

Inacceptable pour un mot de passe.

post

Les données passent dans le **corps de la requête**.

Invisibles dans la barre d'adresse.

Le choix par défaut dès qu'on crée ou modifie quelque chose.

2. Les champs de saisie

<input> et son attribut type

Une seule balise, auto-fermante, dont le comportement change **entièrement** selon son type.

type	Usage	Contrôle automatique
text	Texte libre sur une ligne	aucun
email	Adresse e-mail	présence d'un @ et d'un domaine
password	Mot de passe	aucun ; masque les caractères
number	Valeur numérique	chiffres, bornes min / max
date	Date	calendrier, format garanti
url	Adresse web	format d'URL
tel	Numéro de téléphone	aucun (les formats varient trop)
checkbox	Case à cocher	—
radio	Choix unique dans un groupe	—
hidden	Donnée transmise sans être affichée	—

Le bon type améliore surtout le téléphone

Un gain gratuit

Sur mobile, le clavier affiché s'adapte au type du champ :

- `number` → pavé numérique
- `email` → clavier avec la touche @
- `date` → sélecteur de calendrier
- `tel` → clavier de téléphone

Choisir le bon type est le moyen le **moins coûteux** d'améliorer un formulaire : un mot à changer, aucun code à écrire.

Les autres champs

```
<!-- Texte long, sur plusieurs lignes -->
<textarea id="message" name="message" rows="6" maxlength="500"></textarea>

<!-- Liste deroulante -->
<select id="categorie" name="categorie">
  <option value="general">General</option>
  <option value="aide">Aide</option>
</select>

<button type="submit">Publier</button>
```

Balise	Rôle
<textarea>	Jamais auto-fermante : le contenu initial se place entre les deux balises
<select> / <option>	C'est l'attribut value de l'<option> qui est transmis, pas le texte affiché
<button>	type="submit" envoie le formulaire ; type="button" ne fait rien sans JavaScript

Le piège du `<button>` sans type

Le comportement par défaut

À l'intérieur d'un `<form>`, un `<button>` sans attribut `type` vaut `submit` : il **envoie le formulaire et recharge la page**.

Ce que ça donne en semaine 7

C'est la cause d'une bonne moitié des « mon bouton ne marche pas » : le code JavaScript s'exécute bien, mais la page se recharge dans la foulée et efface tout.

Précise toujours le `type`, même quand il semble évident.

Cases à cocher et boutons radio

```
<!-- Case à cocher : chaque case est indépendante -->
<input type="checkbox" id="news" name="news">
<label for="news">Recevoir la newsletter</label>

<!-- Boutons radio : même name = même groupe -->
<input type="radio" id="pub" name="visibilite" value="public" checked>
<label for="pub">Public</label>

<input type="radio" id="priv" name="visibilite" value="prive">
<label for="priv">Privé</label>
```

Une **case à cocher** autorise plusieurs sélections indépendantes ; un **bouton radio** impose un choix unique dans un groupe.

La règle des boutons radio

Même `name` pour tout le groupe

Pour qu'un groupe de boutons radio fonctionne — un seul sélectionnable à la fois — tous ses boutons doivent partager le **même** `name`.

Attribut	Dans un groupe de radios
<code>name</code>	identique pour tous : c'est lui qui les relie
<code>value</code>	différente pour chacun : c'est l'option choisie
<code>id</code>	unique pour chacun : c'est le lien avec son <code><label></code>

Deux `name` différents produisent deux groupes indépendants — symptôme classique quand « les deux boutons restent allumés ».

3. Les attributs qui encadrent la saisie

Le tableau des attributs

Attribut	Effet
name	Le nom sous lequel la valeur sera transmise au serveur. Sans lui, le champ n'est pas envoyé
id	Identifiant unique dans la page ; sert au <label>, au CSS et à JavaScript
value	Valeur initiale du champ (ou, pour radio et checkbox, la valeur envoyée)
placeholder	Texte d'exemple grisé, qui disparaît dès la première frappe
required	Le champ doit être rempli pour que le formulaire parte
maxlength / minlength	Bornent le nombre de caractères
min / max / step	Bornent une valeur numérique ou une date
pattern	Impose un format via une expression régulière
readonly	Champ visible et transmis , mais non modifiable
disabled	Champ inactif et non transmis au serveur

name et id : deux mondes

	id	name
Sert à	désigner l'élément dans la page	nommer la valeur pour le serveur
Utilisé par	le <label>, le CSS, JavaScript	le formulaire au moment de l'envoi
Unicité	unique dans la page	partagé par les radios d'un groupe
Si absent	impossible de relier un label	le champ n'est pas transmis

La confusion la plus tenace de la semaine

Les deux attributs se ressemblent, portent souvent la **même valeur**, et ne servent pourtant à rien de commun.

Ce que le serveur recevra en semaine 15

Au moment de l'envoi

Le navigateur assemble les champs en une suite de paires `nom=valeur`. Pour le formulaire de création de sujet :

```
pseudo=alice&titre=Bonjour&categorie=general&message=Salut
```

Chaque clé est un attribut `name`

En semaine 15, le serveur Express lira exactement ces clés : `req.body.pseudo`, `req.body.titre...`

Un champ sans `name` est **absent de la liste** : le serveur ne saura même pas qu'il a existé. C'est pour cela qu'on nomme les champs **dès maintenant**, alors que rien ne les lit encore.

pattern : imposer un format

```
<label for="pseudo">Pseudo</label>
<input type="text" id="pseudo" name="pseudo"
  pattern="[A-Za-z0-9]{3,15}"
  title="3 a 15 caracteres : lettres et chiffres uniquement"
  required>
```

pattern sans title est inutilisable

Si la saisie ne correspond pas au motif, le navigateur affiche un message générique du type « Veuillez respecter le format demandé » : l'utilisateur ne sait **pas quoi corriger**.

L'attribut `title` fournit le texte affiché à la place. Il n'est pas décoratif — sans lui, le champ devient un piège.

4. La validation native et ses limites

Ce que le navigateur fait tout seul

Un champ `required` vide ou un `type="email"` mal formé **bloquent l'envoi** et déclenchent une bulle d'avertissement — sans une ligne de JavaScript.

Ce qu'elle apporte

- Un retour **immédiat**, avant tout aller-retour avec le serveur
- Des messages traduits dans la langue du navigateur, sans effort
- Un premier filtre contre les saisies manifestement erronées

Le test à faire sur chaque formulaire du TP

Laisse un champ `required` vide et clique sur le bouton d'envoi. Le navigateur doit refuser, placer le curseur dans le champ fautif et afficher un message.

Une validation côté navigateur ne protège rien

C'est une aide à la saisie, pas une sécurité

Elle s'exécute sur la machine du **visiteur**, qui garde la main dessus.

- Ouvrir les outils de développement et supprimer l'attribut `required` suffit à envoyer un formulaire vide.
- On peut même ne pas utiliser de navigateur du tout : un outil en ligne de commande envoie une requête directement au serveur, sans jamais avoir vu le formulaire.

Conséquence, valable pour toute l'année

Tout contrôle effectué dans le navigateur doit être **refait sur le serveur**. Mise en œuvre en semaine 15 ; en semaine 19, on verra ce qu'un attaquant fait d'un serveur qui a fait confiance au navigateur.

Les quatre niveaux de contrôle

Niveau	Ce qu'il assure	Vu en
Navigateur (HTML5)	Confort de saisie, erreurs évidentes	semaine 3
JavaScript (client)	Messages sur mesure, règles combinées	semaine 7
Serveur (Express)	La seule barrière fiable	semaine 15
Base de données	Intégrité des données stockées	semaine 8

Les trois premiers niveaux se complètent. Un seul est obligatoire.

5. Structurer un formulaire

<fieldset> et <legend>

```
<fieldset>
  <legend>Votre message</legend>

  <label for="titre">Titre du sujet</label>
  <input type="text" id="titre" name="titre" maxlength="80" required>

  <label for="message">Message</label>
  <textarea id="message" name="message" rows="6" required></textarea>
</fieldset>
```

Balise	Rôle
<fieldset>	Regroupe des champs qui vont ensemble
<legend>	Le titre du groupe ; première balise à l'intérieur du <fieldset>

Indispensable pour un groupe de boutons radio

Ce qu'entend une personne sans écran

Un lecteur d'écran annonce l'étiquette de chaque bouton radio — « Public », « Réservé aux membres » — mais **rien ne dit de quoi il s'agit**.

Avec un `<legend>`

La personne entend « **Visibilité** : Public ». L'information manquante est fournie par le titre du groupe.

Pour un champ isolé, le `<label>` suffit. Pour un **groupe**, il faut les deux.

6. Accessibilité

De quoi parle-t-on ?

Définition — Accessibilité (a11y)

Rendre un site utilisable **par tous**, y compris les personnes en situation de handicap : malvoyance, cécité, daltonisme, difficultés motrices empêchant l'usage d'une souris, troubles cognitifs.

Ce n'est pas une couche que l'on ajoute à la fin : elle se joue à **80 %** dans le choix des balises HTML — donc maintenant.

Trois raisons de s'en préoccuper dès la troisième semaine

- **Des gens.** Une page inaccessible exclut des utilisateurs réels, souvent sans que personne s'en aperçoive.
- **La loi.** Les sites publics suisses sont soumis à des exigences d'accessibilité, et la plupart des appels d'offres publics les imposent.
- **La qualité.** Un HTML accessible est un HTML bien structuré : mieux référencé, plus facile à maintenir.

<label> — le point le plus important de la semaine

```
<!-- Ecriture explicite : for = id (la plus courante) -->
<label for="pseudo">Votre pseudo</label>
<input type="text" id="pseudo" name="pseudo">

<!-- Ecriture implicite : le label enveloppe le champ -->
<label>
  Votre pseudo
  <input type="text" name="pseudo">
</label>
```

Le test d'une seconde

for reprend l'id, jamais le name. Pour vérifier : **clique sur l'étiquette**. Si le curseur se place dans le champ (ou si la case se coche), la liaison fonctionne.

Un placeholder n'est pas un <label>

Quatre raisons de ne pas supprimer les étiquettes

- le texte **disparaît** dès la première frappe — l'utilisateur ne sait plus ce qu'il remplit ;
- son contraste est volontairement faible, donc difficile à lire ;
- plusieurs lecteurs d'écran ne l'annoncent pas ;
- il n'y a plus de zone cliquable pour atteindre le champ.

La bonne répartition

Le placeholder donne un **exemple** de saisie (ex : alice).

Le <label> dit **ce qu'est** le champ.

Les deux, pas l'un à la place de l'autre.

Le texte alternatif dépend du rôle de l'image

Rôle de l'image	Que mettre dans alt
Elle porte une information	Ce que l'image dit : <code>alt="Trois sujets sans réponse depuis lundi"</code>
Elle est un lien ou un bouton	La destination ou l'action : <code>alt="Retour à l'accueil"</code>
Elle est décorative	Un alt vide : <code>alt=""</code> , pour que le lecteur d'écran l'ignore

Le test du téléphone

Imagine que tu décrives la page à quelqu'un au téléphone. Ce que tu dirais de l'image, c'est le alt. Ce que tu passerais sous silence parce que c'est un simple élément de décor, c'est un alt="".

Définition — Rapport de contraste

Il compare la luminosité d'un texte et de son fond. Il va de **1 :1** (invisible) à **21 :1** (noir sur blanc).

Seuil de référence, niveau **AA** des règles WCAG : **4,5 :1** pour le texte courant, **3 :1** pour les grands titres.

La couleur ne doit jamais être la seule information

Un champ en erreur signalé **uniquement** par une bordure rouge est invisible pour une personne daltonienne — environ 8 % des hommes.

Double systématiquement la couleur d'un second signal : un texte (« Ce champ est obligatoire »), une icône, ou un soulignement. Vaut aussi pour les liens dans un paragraphe.

Touche	Action
Tab	Passer à l'élément interactif suivant
Maj + Tab	Revenir au précédent
Entrée	Activer un lien ou envoyer un formulaire
Espace	Cocher une case, activer un bouton
Flèches	Se déplacer dans un groupe de radios ou une liste déroulante

Le test des trente secondes

Clique dans la barre d'adresse, puis parcours toute la page avec Tab sans jamais toucher la souris. Trois choses doivent être vraies : tu **vois toujours** où tu te trouves ; l'ordre suit la **lecture naturelle** ; tu peux atteindre **tous** les liens, champs et boutons.

Deux règles à retenir pour la suite

L'ordre de tabulation suit l'ordre du **code**

Pas l'ordre visuel. Si le CSS déplace un élément (semaines 4 et 5), la navigation clavier continuera de suivre le HTML.

Une raison de plus d'écrire le HTML dans un ordre logique dès maintenant.

```
/* A ne jamais écrire */  
button:focus { outline: none; }
```

Ne supprime jamais le cadre de focus

On trouve souvent cette ligne dans du CSS repris en ligne : à elle seule, elle rend un site inutilisable au clavier. Si le cadre par défaut ne plaît pas, on le **remplace** par un autre style visible — on ne l'efface pas.

Outil	Ce qu'il apporte
Onglet <i>Accessibilité</i> des DevTools	L'arbre tel que le perçoit un lecteur d'écran, et les contrastes insuffisants
Lighthouse (Chrome)	Un rapport noté, avec la liste des problèmes détectés
<code>validator.w3.org</code>	Les <code>alt</code> manquants, les <code>for</code> orphelins, les <code>id</code> en double
Le clavier	Le test le plus rapide, et celui qu'aucun outil ne remplace

Aucun outil automatique ne suffit

Un rapport Lighthouse à 100 ne signifie pas qu'une page est accessible : une machine sait vérifier qu'un `alt` **existe**, pas qu'il **décrit correctement** l'image. Ces outils repèrent les erreurs mécaniques ; le reste relève du jugement.

Formulaire

- Chaque `<input>` a un `<label>` relié par `for / id`
- Chaque champ à transmettre a un `name`
- Les groupes de radios sont dans un `<fieldset>` avec `<legend>`
- Le placeholder complète le label, il ne le remplace pas

Page

- Chaque `<img>` a un `alt` adapté à son rôle
- L'information n'est jamais portée par la seule couleur
- La page se parcourt entièrement au clavier, focus visible
- La page passe le validateur W3C sans erreur

Ce qu'on a vu

- `<form>`, `<input>`, `<textarea>`, `<select>`, `<button>`
- Le type change tout : contrôle et clavier mobile
- `name` pour le serveur, `id` pour la page
- La validation native n'est pas une sécurité
- `<label>`, `<fieldset>`, `<legend>`
- `alt`, contraste, navigation au clavier

À faire pour la semaine 4

- Les **quatre formulaires** du MiniForum terminés (fiche TP)
- Chaque page passe `validator.w3.org` sans erreur
- Le site est entièrement parcourable au clavier
- Le tout poussé sur GitHub, en plusieurs commits
- Faire le quiz Moodle

Où en est le projet

Le MiniForum contient désormais l'intégralité de ses formulaires : création de sujet, réponse à un sujet, inscription et connexion.

Ce qui ne bougera plus

Ils ne font encore rien — mais leur structure ne changera plus jusqu'à la fin du cours.

Les `name` choisis cette semaine sont ceux que le serveur Express lira en **semaine 15**.

Les `id` sont ceux que JavaScript utilisera dès la **semaine 7**.

Trois semaines de HTML sont terminées. La semaine 4 ouvre le **CSS** : la structure va enfin recevoir sa mise en forme.

Des questions ?

Prochaine étape : la fiche TP — les quatre formulaires du MiniForum